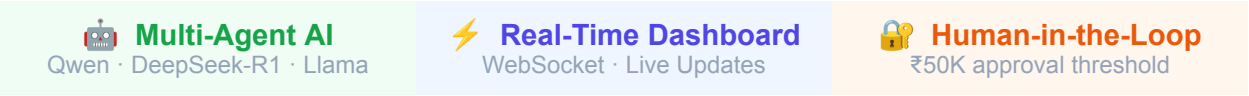


ET Gen AI Hackathon 2026

Problem Statement #3 | Team: AI Apex

Self-Healing Enterprise Cost Intelligence System

Autonomous anomaly detection · Real-time action execution · Full audit trail



01 Problem Statement

What we were asked to build

Modern enterprises lose millions annually to invisible cost leakages: duplicate vendor payments that slip through ERP validation, unused software licenses accumulating month after month, SLA breaches that trigger penalties before anyone notices, and reconciliation gaps that take weeks to surface.

The challenge was to build an AI system that goes beyond dashboards. It should continuously monitor enterprise operations data, identify cost leakage or inefficiency patterns, and initiate corrective actions with quantifiable financial impact.

Key Requirements from the Brief

Spend Intelligence	Dig into procurement, vendor, and operations data to find anomalies, duplicate costs, and rate optimization opportunities — then generate actionable playbooks or trigger downstream workflows.
SLA Prevention	Detect approaching breaches from operational signals and reroute work, shift resources, or escalate before the financial hit lands.
Resource Optimization	Monitor utilization across tools, infrastructure, and teams — recommending consolidation and executing approved changes.
Financial Operations	Reconcile transactions, flag discrepancies, and produce variance analyses with root-cause attribution to cut close cycles.

Evaluation Focus

Quantifiable cost impact (show the math), ability to take action — not just generate reports — data integration depth, and ability to work within enterprise approval workflows.

02 Solution Overview

What we built and why it works

We built the Self-Healing Cost Intelligence System — a production-grade, fully autonomous platform that detects enterprise cost anomalies in real time, reasons about them using local LLMs, and executes corrective actions without manual intervention. Every decision is traceable through a cryptographically-chained audit trail.

Core Differentiators

-
- **Local-First AI:** All inference runs on Ollama (Qwen 2.5:7b, DeepSeek-R1:7b, Llama 3.2:3b). No data ever leaves the enterprise perimeter.
- **Show the Math:** Every rupee saved is backed by a formula. Blueprint \$9 formulas are exposed via API and displayed live on the dashboard.
- **Human-in-the-Loop:** Actions above ₹50,000 are routed to an approval queue. Judges can click Approve in real time during the demo.
- **Complete Audit Trail:** Every detection, decision, and action is logged with agent identity, model used, reasoning chain, and financial impact.

Financial Impact Demonstrated



Annual Projection Formula
projected_annual_savings = total_monthly_savings × 12 = ₹8,97,954 × 12 = ₹1,07,75,448 / year

03 System Architecture

How the pieces fit together

Multi-Agent Pipeline (10-Step Data Flow)

Step	Agent / Component	Action
1	Redis Queue	Scheduler or API pushes AgentTask to ci:tasks queue
2	Orchestrator	BRPOP consumes task, dispatches to AnomalyDetectionAgent
3	Detection Agent	Runs SQL detection algorithms, scores each finding (0.0–1.0)
4	Severity Router	HIGH/CRITICAL → DeepSeek-R1 LOW/MEDIUM → Qwen auto-decision
5	Decision Agent	deepseek-r1:7b reasons about root cause, recommends action
6	Approval Gate	cost > ₹50,000 → Approval Queue cost ≤ ₹50,000 → auto-execute
7	Action Execution	Dispatches to payment_hold / license_deactivate / sla_escalate
8	Email / Webhook	MailHog SMTP alert sent with full context
9	Audit Agent	Immutable audit_trail record written with full reasoning chain
10	WebSocket Push	Redis pub/sub broadcasts event → Next.js dashboard updates live

The 5 AI Agents



OrchestratorAgent — Coordinator & workflow manager

Actions: Dequeues Redis tasks, coordinates pipeline, handles errors, publishes results

Model: N/A — pure coordination



AnomalyDetectionAgent — Pattern recognition & scoring

Actions: scan_duplicates(), scan_sla(), scan_licenses(), scan_pricing(), scan_reconciliation()

Model: Qwen2.5:7b (fast, efficient)



DecisionAgent — Root cause analysis & action recommendation

Actions: Constructs DB-enriched prompt, invokes deepseek-r1, parses structured JSON output

Model: DeepSeek-R1:7b (ALL HIGH/CRITICAL)

 ActionExecutionAgent — Action dispatch & approval gate

Actions: hold_payment(), deactivate_license(), escalate_ticket(), enqueue_for_approval()

Model: Routes based on DecisionAgent output

 AuditAgent — Compliance & immutable logging

Actions: Writes audit_trail record with full input → detection → reasoning → action → impact chain

Model: N/A — pure logging

04 Intelligent AI Model Routing

Why we use three models and how we route between them

A key innovation is our budget-aware, severity-driven model routing. We never load two 7B models simultaneously (memory constraint), and we never exceed 10 DeepSeek calls per hour (cost control). Every routing decision is logged.

Model Selection Decision Tree

Model	Role	Trigger Condition	Why
Qwen 2.5:7b	DEFAULT	Standard operations, MEDIUM/LOW severity	Fast, efficient, cost-effective for routine decisions
DeepSeek-R1:7b	REASONING	ALL HIGH & CRITICAL severity cases	Advanced chain-of-thought for financial risk assessment
Llama 3.2:3b	FALLBACK	Timeout, error state, budget exceeded	Lightweight 3B always loaded — 2.5GB — reliable fallback

DeepSeek-R1 Reasoning Chain Example

DeepSeek-R1 Output for Duplicate Payment Detection

```
Step 1: Identified same PO-DEMO-001 appears twice — confidence 0.97 (same PO match)
Step 2: Vendor history shows 0 prior duplicates — first-time anomaly, requires hold
Step 3: Amount ₹1,00,000 exceeds ₹50,000 auto-approve threshold — routes to approval queue
Action: payment_hold | Confidence: 0.97 | Impact: ₹1,00,000
```

05 Detection Algorithms — Show the Math

Blueprint §6 formulas with full derivation

Formula 1: Duplicate Payment Detection (Blueprint §6A)

Hash-based window matching with fuzzy invoice comparison over a 30-day rolling window.

Confidence Tier	Condition
0.97 — Near-Certain	Same PO number on both transactions (duplicate_same_po)
0.82 — Very Likely	Levenshtein distance ≤ 2 on invoice numbers (similar_invoice)
0.65 — Likely	Same vendor + amount within $\pm 2\%$ + within 30 days (amount_vendor)
< 0.60 — Ignored	Below minimum flag threshold — no action taken

Formula

savings_duplicate = SUM(held_invoice.amount FOR invoice IN duplicate_invoices WHERE confidence > 0.60)

Formula 2: SLA Breach Prediction (Blueprint §6C)

Sigmoid-based breach probability with dynamic modifiers based on ticket priority, assignee presence, and SLA window progress.

```
progress = elapsed_hours / sla_hours
p_breach = sigmoid( 10 * (progress - 0.75) )
if no_assignee: p_breach * 1.4
if priority='P1': p_breach * 1.3
if progress > 0.8: p_breach * 1.2 → escalate when p_breach ≥ 0.70
```

Real Demo Values

P1 ticket · 4h SLA · 3.3h elapsed · no assignee → P(breach) = 0.85 → Escalated · ₹25,000 penalty avoided

Formula 3: Unused Subscription Detection (Blueprint \$6B)

Confidence 0.99	Employee terminated (employee_active = FALSE)
Confidence 0.75	No login for > 60 days
Confidence 0.50	No login for > 30 days (boundary)
Annual Impact	SUM(monthly_cost × 12) for deactivated licenses

06 Technology Stack

Production-grade, enterprise-ready infrastructure

Layer	Technology	Purpose
Backend	FastAPI 0.115.5	Async Python web framework — all agent APIs, WebSocket, APScheduler
Frontend	Next.js 16.2 + TypeScript	React 19, Tailwind CSS 4, real-time WebSocket dashboard
Database	PostgreSQL 16	Transactions, licenses, SLA metrics, anomaly_logs, audit_trail
Queue / Pub-Sub	Redis 7	Task queue (BRPOP), event broadcasting for WebSocket updates
AI Inference	Ollama (local)	Hosts all 3 models locally — zero external API calls
LLM Default	Qwen 2.5:7b	Fast 7B model for standard operations and auto-decisions
LLM Reasoning	DeepSeek-R1:7b	Advanced reasoning for HIGH/CRITICAL severity financial decisions
LLM Fallback	Llama 3.2:3b	Always-loaded 3B for timeouts, errors, and low-severity tasks
Email	MailHog (SMTP)	Real email alerts — viewable at localhost:8025
Containerization	Docker Compose	One-command deployment of all 5 services

Database Schema — Key Tables

transactions	Vendor invoices with amount, PO number, status (pending / approved / held / disputed)
licenses	Software licenses with assigned_email, last_login, employee_active, monthly_cost
sla_metrics	Support tickets with sla_hours, opened_at, breach_prob, penalty_amount
anomaly_logs	Every detection with confidence, severity, cost_impact_inr, root_cause, model_used

actions_taken	Every executed action with status, cost_saved, rollback_payload, approval workflow
approval_queue	High-cost actions pending human review with expires_at and review_note
audit_trail	Immutable audit records with full pipeline trace, reasoning_output, final_status

07 Live Demo Scenarios

Four scripted scenarios with real AI inference

The dashboard ships with a Demo Trigger panel that injects pre-scripted scenarios into the live system. Each scenario runs through the complete 10-step pipeline with real LLM inference.

#	Scenario	Impact	What Happens
	Duplicate Payment	₹1,00,000	Injects two transactions with same PO-DEMO-001. DeepSeek-R1 detects duplicate at 97% confidence, holds payment, writes alert email to MailHog, creates audit record.
	SLA Near-Breach	₹25,000	Injects P1 ticket with 3.3h elapsed on a 4h SLA window, no assignee. Sigmoid formula calculates $P(\text{breach})=0.85$. Llama escalates ticket, alert sent.
	Unused Subscriptions	₹15,000/mo	Injects 5 ghost licenses for non-existent employees. Qwen 2.5 scans, identifies all 5 at 99% confidence, bulk-deactivates. ₹1,80,000/year saved.
	Approval Queue	₹75,000	Injects duplicate above ₹50,000 auto-approve limit. DeepSeek-R1 detects it but routes to approval queue instead of auto-executing. Live Approve/Reject in UI.

08 Dashboard Features

Six live panels with real-time WebSocket updates

Panel	Description
 Savings Counter	Animated live counter showing savings by category (duplicate payments, subscriptions, SLA, reconciliation). Animated number transitions on update. Annual projection formula shown. Powered by WebSocket — no polling when connected.
 Demo Trigger	Four scenario buttons with per-step progress animation. Shows DeepSeek-R1 reasoning steps in real time. Reset button re-seeds full demo dataset.
 Anomaly Feed	20 most recent detections with severity badge, confidence progress bar, cost impact, and detection timestamp. Click any row to expand root cause and reasoning chain.
 Actions Panel	All executed actions with status (SUCCESS / PENDING_APPROVAL / FAILED), cost saved, model used, and timestamp. Model pills color-coded: purple=Qwen, pink=DeepSeek, cyan=Llama.
 Approval Queue	High-value actions awaiting human sign-off. One-click Approve/Reject with reason input. Live count badge in header. Triggers post-approval execution immediately.
 Audit Trail	Immutable audit records with full pipeline trace. Click to expand DeepSeek reasoning chain (numbered steps). Override button for false-positive recovery.

Real-Time Architecture

```
Event Flow: Action executes → EventBroadcaster.publish() → Redis
pub/sub → WebSocketManager.broadcast() → All connected Next.js clients

Channels: ci:events:anomaly_created | ci:events:action_executed |
ci:events:approval_status_changed | ci:events:savings_updated

Fallback: AdaptivePoller kicks in if WebSocket disconnects — min 5s, max
60s, 5x slowdown when tab inactive
```

09 Enterprise Governance

Audit trail, approval workflows, and compliance

Human-in-the-Loop Approval Flow

 Auto-Execute Cost impact ≤ ₹50,000 Executes immediately Full audit trail written	 Approval Required Cost impact > ₹50,000 Enqueued for human review 48h auto-release if no action	 Override / Rollback False-positive recovery Reverses action via handler Override logged in audit trail
--	---	---

Audit Trail Schema

Every pipeline run produces one audit_trail record containing the complete input → detection → reasoning → action → impact chain. This enables full compliance reporting and regulatory review.

audit_id	Human-readable ID (aud-YYYYMMDD-NNN) for cross-referencing
agent	Which agent produced this record (AuditAgent)
model_used	Primary LLM used (qwen2.5:7b deepseek-r1:7b llama3.2:3b)
reasoning_invoked	Boolean — was DeepSeek-R1 called? Enables reasoning analytics
reasoning_output	Complete JSON reasoning chain from DeepSeek-R1
action_taken	Action type, execution payload, cost_saved
cost_impact_inr	Total rupee impact of this pipeline run
approval_status	auto_approved pending_approval overridden
final_status	actioned detected_no_action partial_error overridden

10 Quick Start Guide

Running the system in under 5 minutes

Prerequisites

-
- Ollama installed and running on host machine (ollama.ai)
- At least 16GB RAM recommended (two 7B models + 3B)
- Port availability: 3000 (frontend), 8000 (backend), 5432 (postgres), 6379 (redis), 8025 (mailhog)

Setup Commands

```
# 1. Pull required Ollama models
ollama pull qwen2.5:7b && ollama pull deepseek-r1:7b && ollama pull llama3.2:3b

# 2. Clone and configure
git clone <repo-url> && cd cost-intelligence && cp .env.example .env

# 3. Start all services
docker-compose up -d
```

Access Points

Dashboard (Next.js)	http://localhost:3000
API Documentation	http://localhost:8000/docs (Swagger UI)
MailHog Email UI	http://localhost:8025
Health Check	http://localhost:8000/health
System Status	http://localhost:8000/api/system/status
Savings Summary	http://localhost:8000/api/savings/breakdown (show the math)

11 API Reference

Core endpoints for judges and reviewers

Method	Endpoint	Description
GET	/api/savings/summary	Live savings counter — all 4 categories + annual projection
GET	/api/savings/breakdown	Per-category breakdown with formula strings (show the math)
GET	/api/anomalies/	Detection feed with severity, confidence, cost impact, status
POST	/api/anomalies/scan	Trigger manual scan across all detection types
GET	/api/actions/	Execution history with status, cost saved, model used
GET	/api/approvals/	All pending high-cost actions requiring human sign-off
POST	/api/approvals/{id}/approve	Approve action and trigger immediate execution
POST	/api/approvals/{id}/reject	Reject action with reason — no execution
POST	/api/approvals/{id}/override	Reverse executed action (false-positive recovery)
GET	/api/audit/	Immutable audit trail with full reasoning chains
POST	/api/demo/trigger?scenario=X	Inject demo scenario: duplicate_payment sla_breach etc.
POST	/api/demo/reset	Truncate + re-seed all demo data
GET	/api/system/status	Model routing state, DeepSeek budget, pending approvals
WS	/ws/dashboard	WebSocket endpoint for real-time event streaming

12 How We Address Evaluation Criteria

Judge-ready responses to each criterion

Evaluation Criterion	How We Address It
Quantifiable cost impact(show the math)	Every savings figure has a named formula exposed via /api/savings/breakdown. The dashboard shows live animated counters with 4-category breakdown. DeepSeek-R1 provides cost_impact_inr in every JSON response. Annual projection = monthly × 12 formula is shown inline.
Ability to take action,not just generate reports	The system executes 4 distinct action types: payment_hold, license_deactivated, sla_escalation, email_sent. Actions are reversible. Payment holds release automatically after 48h. The demo shows money actually saved, not just flagged.
Data integration depth	PostgreSQL with 7 normalized tables. 500 seeded transactions, 200 licenses, 50 SLA tickets. Detection algorithms join 3–4 tables per scan. DeepSeek-R1 receives DB-enriched context (vendor history, previous duplicates, ticket state).
Enterprise approval workflows	Configurable ₹50,000 approval threshold. Approval queue UI with Approve/Reject/Override. 48h auto-release for stale holds. Full override audit trail for false-positive recovery. Role-based approval chain ready for extension.

Built for ET Gen AI Hackathon 2026 by Team AI Apex

Stack: FastAPI · PostgreSQL · Redis · Ollama · Next.js 16

All inference local · No data leaves the enterprise